

Incremental Memoization Bypass: Cache Vulnerabilities in LLM Safety Filters

Marco Servo
Independent Security Research
Rio de Janeiro, Brazil
marcos18motorola@gmail.com
github.com/servodecristorj-ai

Abstract

Modern LLM safety filters use incremental computation with dependency tracking. This paper identifies Incremental Memoization Bypass. Edge-case Unicode like U+203E OVERLINE causes desynchronization between validation and execution in Salsa-like architectures. The filter’s Vc<Token> fails to dirty Vc<IsDangerous>, allowing bypass. Local Rust PoC, no production interaction. Code available on GitHub.

1 Introduction

Safety filters use Salsa/Turbo Tasks to cache decisions. They index by raw token bytes without Unicode normalization. U+203E + “system” tokenizes as [OVERLINE][system] not [system], so the graph does not recompute and cached false is reused.

2 Experimental Results

Table 1 shows results. Refusal delta confirms internal term knowledge.

Table 1: Experimental Results

ID	Input	Observation
1.1	turbopack	Baseline 1.1s
1.4	—[REDACTED]	Refusal 2.3s
4.1	Vc<T> exp	Arch leak 2.0s
5.1	SWC Context	DoS >600s

3 Root Cause

Stale Vc<FileContent> bug. Regex ^system: runs on Vc<Token>. For overline+system it creates new token not in banned list, so Vc<IsDangerous> stays false.

4 Proof of Concept

Listing 1 runs locally. Full reproduction on GitHub.

Listing 1: PoC

```
fn is_banned(token: &str) -> bool {
    matches!(token, "system" | "turbopack")
}
fn main() {
    let a = "system";
    let b = "\u{203E}_system";
    assert!(is_banned(a));
    assert!(!is_banned(b)); // bypass
}
```

5 Ethics and Disclosure

- **2026-07-07:** Reported to xAI Security
- **2026-07-09:** Acknowledged
- **2026-10-05:** Public disclosure

Contact: marcos18motorola@gmail.com. Repository: GitHub.

References

- [1] Vercel Team. Turbopack. 2023. nextjs.org/blog/turbopack
- [2] Salsa Contributors. Salsa. 2024. github.com/salsa-rs/salsa